# Progress Report

Discrete State-Space Models and Expectation-Maximization for Cell-State Trajectories

Arnab Kumar Ghosh · Supervisor: Prashant Singh · MSc Computer Science, Uppsala University · Weeks 3–4 · October 11, 2026

## Overview

In weeks 3–4 I moved from the model definition to the algorithms. The open question from the last report was how to build the observation sequence. I discussed it with the supervisor on 22 September. I still have questions about it (Section 7). Progress against the plan from the last report:

- Forward and backward algorithms with per-step scaling (Rabiner Problem 1): **done**, on paper and in code.
- Posteriors $\gamma_t(i)$ and $\xi_t(i,j)$, i.e. the E-step: **done** in code.
- Baum–Welch re-estimation (Rabiner Problem 3) for discrete emissions: **done** in code, with checks.
- Sequence definition: discussed with the supervisor; still open, see Section 7.
- Bilmes (1998) and the Gaussian-emission M-step: **not yet**.
- Data received from the biology group, with a read-me describing the dataset.
- Read the previous student's internship report once; my current understanding is in Section 6.
- Started reading the paper behind the dataset: Amoedo-Leite et al., *Nature Cardiovascular Research* 3, 685–700 (2024).

The toy implementation is a small NumPy package (`hmm.py` for the algorithms, `checks.py`, `experiments.py`, `plots.py`): $N = 3$ states, $M = 4$ discrete symbols, known parameters $\lambda = (A, B, \pi)$, a sampler to generate sequences, and one function per algorithm. The code is in the GitHub repository `https://github.com/bdtiger/open-course-on-topics`. The rest of this report summarises what I understand so far and how each part connects to the project.

## 1 The forward algorithm (Rabiner, Problem 1)

### What I learned

The hidden state at time $t$ is $s_t$, the observation sequence is $O = o_1 \cdots o_T$, and the model is $\lambda = (A, B, \pi)$ with

$$\begin{aligned} A[i,j] &= P(s_{t+1} = j \mid s_t = i), && \text{transition: from state } i \text{ to state } j, \\ B[j,k] &= P(o_t = k \mid s_t = j), && \text{emission: state } j \text{ produces symbol } k, \\ \pi_i &= P(s_1 = i). && \text{start in state } i. \end{aligned}$$

In the code these are the arrays `A`, `B` and `pi`.

The forward variable is the probability of seeing the first $t$ observations and ending in state $i$:

$$\alpha_t(i) = P(o_1 \cdots o_t,\ s_t = i \mid \lambda).$$

It is filled in one time step at a time, and the last row gives the likelihood of the whole sequence:

$$\begin{aligned} \alpha_1(i) &= \pi_i B[i, o_1], \\ \alpha_t(j) &= \Big[\sum_{i=1}^{N} \alpha_{t-1}(i) A[i,j]\Big] B[j, o_t], \\ P(O \mid \lambda) &= \sum_{i=1}^{N} \alpha_T(i). \end{aligned}$$

The sum inside the bracket is "all the ways to arrive in state $j$", and the factor $B[j, o_t]$ is "and then see the new observation". This replaces the sum over all $N^T$ state paths with a table of $N \times T$ numbers.

### In the project

$P(O \mid \lambda)$ is the likelihood of the data under the current parameters. This value must never decrease during Baum–Welch. So the forward algorithm is also the check that training works.

## 2 The backward algorithm and the posteriors (E-step)

### What I learned

The backward variable is the probability of the rest of the sequence, given that the state at time $t$ is $i$:

$$\beta_t(i) = P(o_{t+1} \cdots o_T \mid s_t = i, \lambda).$$

It is filled in from the end:

$$\begin{aligned} \beta_T(i) &= 1, \\ \beta_t(i) &= \sum_{j=1}^{N} A[i,j] B[j, o_{t+1}] \beta_{t+1}(j). \end{aligned}$$

Together, $\alpha$ and $\beta$ give the two posteriors needed by EM. $\gamma_t(i)$ is the probability of being in state $i$ at time $t$, given the whole

sequence. $\xi_t(i,j)$ is the probability of being in state $i$ at time $t$ and in state $j$ at time $t+1$, given the whole sequence:

$$\gamma_t(i) = \frac{\alpha_t(i)\,\beta_t(i)}{P(O \mid \lambda)},$$
$$\xi_t(i,j) = \frac{\alpha_t(i)\,A[i,j]\,B[j,o_{t+1}]\,\beta_{t+1}(j)}{P(O \mid \lambda)}.$$

$\alpha$ carries the evidence from the past, $\beta$ the evidence from the future, and $\gamma$ combines both. In the code, every row of $\gamma$ sums to 1, every $\xi_t$ sums to 1, and $\sum_j \xi_t(i,j) = \gamma_t(i)$. These three checks are asserted in `checks.py`.

**In the project**

The per-cell state probabilities (Macrophage, Transitional or Mural-like) from the E-step play the role of $\gamma$. On the toy data, $\arg\max_i \gamma_t(i)$ gives the true state at 73 % of 2000 sampled time steps, well above the 33 % of guessing one of three states at random (`checks.py`). This is how cells could be labelled later.

## 3 Scaling (the numerical issue from the last report)

**What I learned**

Every $\alpha_t(i)$ shrinks like a product of $t$ probabilities, so it underflows to zero for long sequences. The standard fix (Rabiner, Section V.A) is to never build the unscaled $\alpha$. Each row is computed from the *already scaled* previous row, its sum is remembered as the scale factor, and the row is divided by that sum. The first row needs no previous row; for $t = 2, \ldots, T$, given the scaled row $\hat{\alpha}_{t-1}$ from the previous step:

$$\ddot{\alpha}_1(j) = \pi_j B[j,o_1], \qquad \text{first row,}$$
$$\ddot{\alpha}_t(j) = \Big[\sum_{i=1}^{N} \hat{\alpha}_{t-1}(i)\,A[i,j]\Big] B[j,o_t], \qquad \text{one step from the scaled row } t-1,$$
$$c_t = \sum_{j=1}^{N} \ddot{\alpha}_t(j), \qquad \text{scale factor,}$$
$$\hat{\alpha}_t(j) = \frac{\ddot{\alpha}_t(j)}{c_t}, \qquad \text{scaled row, sums to 1,}$$
$$\log P(O \mid \lambda) = \sum_{t=1}^{T} \log c_t.$$

Here $\ddot{\alpha}_t$ is the intermediate row before scaling (`alpha_row` in the code), and $\hat{\alpha}_t$ is the row that is stored. Because $\ddot{\alpha}_t$ is built from a row that already sums to 1, each $c_t = P(o_t \mid o_1 \cdots o_{t-1})$, and the product of all $c_t$ is $P(O \mid \lambda)$. That is why the sum of $\log c_t$ is the log-likelihood. $\beta$ is scaled with the same factors: $\hat{\beta}_T(i) = 1$, and for $t < T$ row $t$ is divided by $c_{t+1}$. The reason: $\beta_t$ covers the future observations $o_{t+1} \cdots o_T$, while $\alpha_t$ already covers $o_1 \cdots o_t$. Then $\hat{\alpha}_t(i)\,\hat{\beta}_t(i)$ sums to 1 at every $t$, so $\gamma$ is unchanged by the scaling, but $\xi$ picks up one extra factor $c_{t+1}$ and needs one extra division:

$$\gamma_t(i) = \hat{\alpha}_t(i)\,\hat{\beta}_t(i), \qquad \text{no division needed,}$$
$$\xi_t(i,j) = \frac{\hat{\alpha}_t(i)\,A[i,j]\,B[j,o_{t+1}]\,\hat{\beta}_{t+1}(j)}{c_{t+1}}, \qquad \texttt{xi[t] /= scale_factors[t+1]} \text{ in the code.}$$

In the code, the forward pass and the backward pass each compute the log-likelihood on their own. The two values agree to machine precision. This is the check that the scaling is consistent.

**In the project**

The implementation uses scaling from the start, as agreed with the supervisor, so the log-likelihood check is always well defined.

## 4 Baum–Welch re-estimation (Rabiner, Problem 3)

### What I learned

The M-step turns the posteriors into new parameters. Every update is a normalised expected count. $\bar{A}[i,j]$ is the expected number of transitions from $i$ to $j$, divided by the expected number of visits to $i$. $\bar{B}[j,k]$ is the expected number of visits to $j$ that emit symbol $k$, divided by the expected number of visits to $j$:

$$\bar{\pi}_i = \gamma_1(i),$$
$$\bar{A}[i,j] = \frac{\sum_{t=1}^{T-1} \xi_t(i,j)}{\sum_{t=1}^{T-1} \gamma_t(i)},$$
$$\bar{B}[j,k] = \frac{\sum_{t:o_t=k} \gamma_t(j)}{\sum_{t=1}^{T} \gamma_t(j)}.$$

Baum–Welch repeats E-step and M-step until the log-likelihood stops increasing. Figure 1 shows it on 2000 toy observations sampled from the true $\lambda$, from 5 random starts. Every curve rises monotonically, which the code asserts. Two things stand out. First, the best start ends slightly *above* the log-likelihood of the true parameters. This is not an error. The true parameters describe the generating process. Baum–Welch tunes the parameters to this one finite sample, noise included, so it can fit the sample a little better. Second, start 2 stops at a lower local optimum. So several random restarts are needed, and the one with the best log-likelihood is kept. Whether the best start also recovers the true $A$, $B$ and $\pi$ is not checked yet; this needs matching the fitted states to the true ones first (plan item 6).

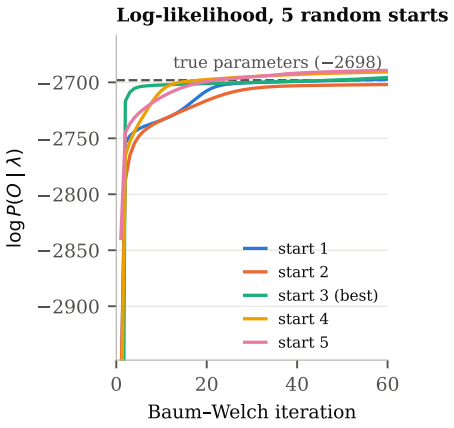



Figure 1: Baum–Welch on 2000 toy observations: log-likelihood per iteration for 5 random starts (200 iterations each, first 60 shown). Dashed line: log-likelihood under the true parameters (−2698.1). The best start (start 3) ends at −2688.0, start 2 at −2697.5. From `experiments.py` and `plots.py`.

### In the project

For a Gaussian-emission HMM only $\bar{B}[j,k]$ changes: it is replaced by the updates for the mean $\mu_j$ and covariance $U_j$ from Bilmes. The population-level model of the project (Sections 6 and 7) will need a different update for $A$, because $A$ is fitted from the change in state proportions between weeks, not from per-cell transitions $\xi_t(i,j)$. The local-optimum behaviour is a warning for the real data: several restarts and a comparison with the known labels will be necessary.

## 5 The dataset

### What I learned

The data arrived as `adata_final.h5ad`, a combination of two single-cell sets (ss2 and ss3). ss2 covers day 21 and healthy tissue. ss3 covers days 7, 14, 21 (few cells) and healthy tissue. So days 7 and 14 come only from ss3, and day 21 comes mostly from ss2. A difference between day 7/14 and day 21 can therefore be partly protocol, not biology. The read-me notes that the pseudotime looks the same on ss2 alone and on ss2+ss3. This suggests the batch effect is not dominant. I will still keep the batch label per cell. Where the healthy cells fit in the model is still open (Section 7). The three states of the model map onto labels that already exist in the data:

| Model state | Label in the data | Note |
|---|---|---|
| Macrophage | `cell_type` = "Macrophages" | Starting population after injury |
| Transitional | `cluster` = 2 (PDGFRb$^+$ macrophages) | Almost absent at day 7 and 14, many at day 21; high unspliced-mRNA fraction, a sign of active reprogramming |
| Mural-like | `cell_type` = "Mural cells", `cluster` = 8 | Marker genes include *Pdgfrb*, *Acta2*, *Myh11*, *Notch3* |

### In the project

The existing labels will be used only as an external check of the fitted states, not as training input. The observation vector $\mathbf{x}$ for each cell will be the expression of the 50 genes listed in the read-me: the top 25 marker genes of cluster 2 (PDGFRb$^+$ macrophages) and the top 25 of cluster 8 (mural cells). A Gaussian over all genes is not feasible. These 50 genes were agreed with the supervisor as the starting point. The genes are the top markers of clusters 2 and 8, and those same clusters define the labels, so agreement between fitted states and labels is partly built in. So there are two levels of observation. One cell is a vector of 50 gene values, and this is what the Gaussian $\mathcal{N}(\mu_j, U_j)$ is applied to. One time step is one day of the experiment, and the observation $o_t$ at that step is the set of all cells measured on that day:

$$\mathbf{x}_1 = (\text{gene}_1, \text{gene}_2, \ldots, \text{gene}_{50}), \qquad \text{one cell},$$
$$o_1 = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_{n_1}\}, \qquad \text{all cells measured on day 7},$$
$$O = o_1\, o_2\, o_3, \qquad \text{day 7, day 14, day 21}.$$

The sequence $O$ is the one from Section 1, and the transition matrix $A$ moves from one day to the next.

## 6 The previous student's report

### What I learned

I read the internship report of the previous student once and need to read it again. What follows is my understanding after that first reading. He built the same three algorithms (forward–backward, Viterbi, Baum–Welch) in Python. To get observations, he ran $k$-means on three marker genes and gave each cell one of three labels. Then he put all cells in one long list, sorted by day, and used that list as the observation sequence. He describes two problems he could not solve:

1. With a random start, Baum–Welch ended in a poor local maximum and the result made no biological sense. So he set $\pi$, $A$ and $B$ by hand to match the paper (Amoedo-Leite et al., 2024). I saw the same thing on my toy data (Section 4). The problem with his fix: if you start from the answer you expect, you mostly get that answer back. Also, any entry set to 0 in $A$ stays 0 forever.
2. His result said 100 % of cells are intermediate on day 7 and 100 % are mural on day 21. According to the previous student's reading of the paper, only about 10 % of macrophages show mural characteristics at day 7 and about 20 % become mural cells by day 21; I have not yet checked these numbers in the paper myself (plan item 5). As far as I understand it, the reason is how the sequence was built. Each cell in the list counts as one time step. So there are thousands of steps, but the state changes only a few times, at the day boundaries. Baum–Welch therefore learns that changing state is almost impossible: $A$ has 0.993, 0.999 and 1.0 on the diagonal. Viterbi then gives the same state to every cell in a day block. So the proportions can only be 0 % or 100 %. This comes from the construction, not from the biology.

### In the project

This supports the direction agreed with the supervisor. In the population-level model, one step of $A$ is one week, not one cell. Cells within a day are a mixture, so proportions can be any value. Emissions are Gaussians on real expression values, not $k$-means labels. Per-cell states come from the E-step probabilities, not from one Viterbi path. I will also use several random restarts and start from the data, not from the expected answer.

## 7 What is still unclear to me

- Sequence definition: scRNA-seq measures each cell only once, so no sequence of individual cells can be built (Section 6). The supervisor's email of 20 September proposes a population-level model instead: one step of $A$ is one week, and the cells on a day are a mixture of the states. I have read the proposal but I do not yet understand how it is written as a likelihood or how it is fitted. Working through it with the supervisor is plan item 2 and my main question for the next meeting.
- The Gaussian-emission M-step (Bilmes): how the $\bar{B}[j,k]$ update above becomes updates for $\mu_j$(Mean) and $U_j$(Covariance), and how to keep $U_j$ well conditioned when a state has few cells.
- Healthy tissue: both ss2 and ss3 contain healthy-tissue cells, but the model starts at "Macrophage after injury" and healthy is not a time point after injury. Should these cells be considered at all, and if so, where do they go in the sequence? I will ask at the next meeting.
- How many random restarts of Baum–Welch are needed on the real data.

## 8 Plan for the next two weeks

1. Read Bilmes (1998): Gaussian mixture example first, then the HMM case. Replace the discrete $B[j,k]$ in the toy code by 1D and then 2D Gaussians. Check that the known means and variances are recovered.
2. Write down the population-level likelihood and its E-step and M-step with the supervisor before fitting anything to the real data.
3. Open `adata_final.h5ad`, check the normalisation, and compute the state proportions per time point.
4. Re-read the previous student's report and check my reading in Section 6 against it.
5. Read Amoedo-Leite et al. (2024), in particular the day 21 analysis and Figure 3e. Look at the published analysis code to see the normalisation.
6. Check parameter recovery of the toy Baum–Welch (best of the random restarts in Section 4) after matching the state labels.
7. Set up the validation agreed in the meeting: simulate cells from a fitted model and compare, gene by gene, the simulated and the real distribution of expression values, and compare the modelled state proportions per week with the observed ones.